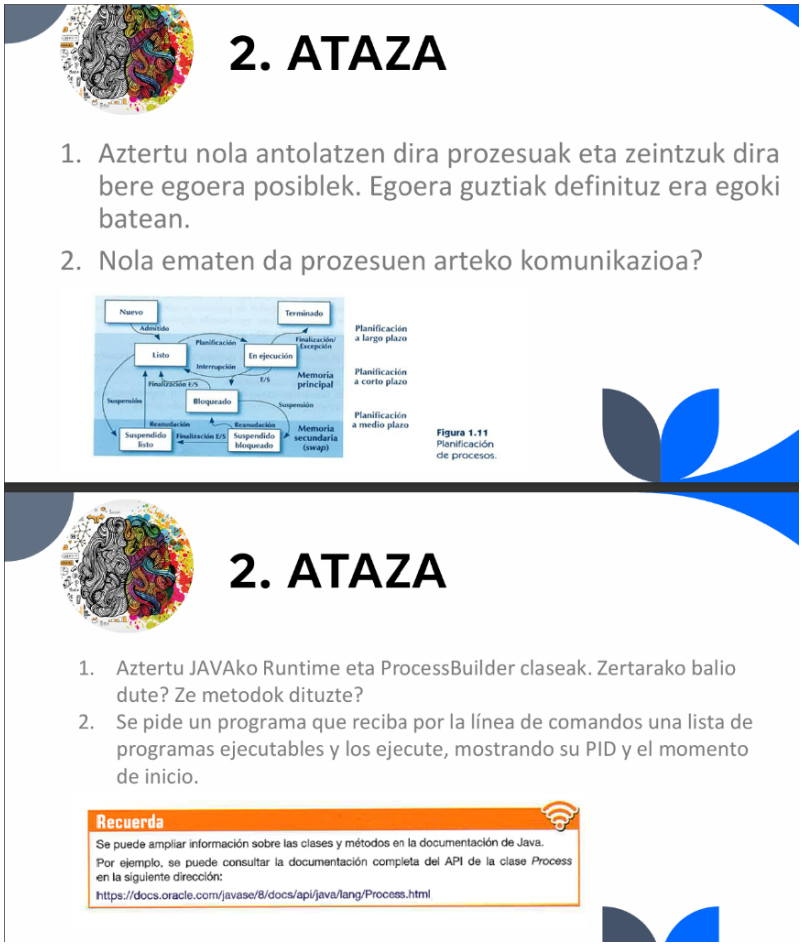
GOIERRI ESKOLA | 0490 - ZERBITZU ETA PROZESUEN PROGRAMAZIOA

2. ATAZA: EBAZPEN TEORIKO ETA PRAKTIKO OSOA

Prozesuen Antolaketa, Bizitza-Zikloa (7 Egoera), Komunikazioa (IPC), Java Runtime & ProcessBuilder eta Exekuzio Programa

Data: 2026-10-06



1. Irudia: 2. Atazako ariketa guztien jatorrizko enuntziatua (2_ataza.png)

> 📌 **DOKUMENTUAREN HELBURUA:** Dokumentu honek enuntziatuko 4 puntu zehatzak erantzuten ditu xehetasun tekniko eta akademiko osoz. Atal bakoitzean teoria sakona, egitura grafikoak, konparaketa-taulak eta IntelliJ IDEA bidez errealitatean egindako exekuzioen pantaila-kapturak txertatu dira.

EDUKIA

1. Prozesuen Antolaketa eta Egoera Posibleak 3
1.1. Prozesuen Antolaketa Sistema Eragilean eta PCB Egitura 3
1.2. Sistema Eragilearen 3 Planifikazio Mailak 4
1.3. Prozesuen 7 Egoera Posibleak eta Bizitza-Zikloa 4
2. Prozesuen Arteko Komunikazioa (IPC) 6
2.1. Zergatik Behar Da IPC? (Memoria-Babesa eta Isolamendua) 6
2.2. IPC Mekanismo Nagusien Azterketa 6
2.3. Sinkronizazioa eta Problemak (Mutex, Deadlock) 6
3. Java Runtime eta ProcessBuilder Klaseak 7
3.1. java.lang.Runtime Klasea 7
3.2. java.lang.ProcessBuilder Klasea 8
3.3. java.lang.Process eta ProcessHandle Klaseak 9
4. PUNTUA: Exekutagarrien Zerrenda Abiarazteko Java Programa 9
4.1. Programaren Arkitektura eta Diseinua 9
4.2. Kode Iturburu Osoa (ProzesuExekutatzailea.java) 9
4.3. IntelliJ IDEA-n Konfigurazioa eta Kodearen Pantaila-Kapturak 11
4.4. Exekuzio Errealaren Emaitzak eta Kontsolako Kapturak 12
4.5. Kasu Praktiko Osagarria: Prozesuak Terminalean Identifikatu eta Hiltzea (tasklist & taskkill) 13
5. ONDORIO NAGUSIAK ETA LABURPENA 14
6. Erreferentzia Bibliografikoak 14

1. Prozesuen Antolaketa eta Egoera Posibleak

Enuntziatua: *Aztertu nola antolatzen dira prozesuak eta zeintzuk dira bere egoera posiblek. Egoera guztiak definituz era egoki batean.*

1.1. Prozesuen Antolaketa Sistema Eragilean eta PCB Egitura

Sistema Eragile moderno baten ikuspuntutik, prozesua ez da diskoan gordetako fitxategi estatikoa (programa edo exekutagarria), baizik eta exekuzio-bidean dagoen entitate aktibo eta dinamikoa. Prozesu bakoitzak baliabide propioak eta isolatuak ditu: CPU denbora, RAM memoria-eremu independentea (helbide-espazio logikoa), irekitako fitxategien taula eta lehentasuna.

Multiprogramazioa ahalbidetzeko, Sistema Eragileak prozesu bakoitza datu-egitura zentral baten bidez erregistratu eta kudeatzen du: PCB (Process Control Block edo Prozesuaren Kontrol Blokea). PCBak prozesuaren 'NAN' edo nortasun-agiri digital gisa jokatzen du.

PCBaren Eremua	Deskribapena eta Funtzioa
PID (Process ID)	Sistema eragileak esleitutako zenbakizko identifikatzaile bakarra (adib. 14250).
Uneko Egoera	Prozesua une horretan non dagoen: Berria, Prest, Exekuzioan, Blokeatua edo Esekia.
Program Counter (PC)	CPUak exekutatu behar duen hurrengo makina-aginduaren memoria-helbidea.
CPU Erregistroak eta SP	Eragiketen datuak gordetzen dituzten CPU erregistroen egoera (AX, BX, Stack Pointer...).
Memoria Kudeaketa	Oinarrizko helbidea, muga-helbidea eta orri-taulak (Memory Protection bermatzeko).
Planifikazio Informazioa	Prozesuaren lehentasuna (nice), quantum balioa eta ilaretako kokapena.
Baliabideen Kontabilitatea	Irekitako fitxategien deskriptoreak, S/I gailuen erabilera eta kontsumitutako CPU denbora.

Testuinguru-Aldaketa (Context Switch): CPUak prozesu baten exekuzioa eten eta beste bati ekiten dionean, unean martxan dagoen prozesuaren CPU erregistroak bere PCBean gordetzen dira, eta hurrengo prozesuaren balioak CPUra kargatzen dira. Eragiketa honek gainkarga (overhead) sortzen du, baina funtsezkoa da multiataza bermatzeko.

1.2. Sistema Eragilearen 3 Planifikazio Mailak

Enuntziatuko 1.11 irudiak erakusten duen bezala, sistema eragile batek prozesuak hiru planifikazio-maila nagusitan kudeatzen ditu:

Planifikazio Maila	Osagaia / Maiztasuna	Helburua eta Zeregina
Epe Luzekoa (Largo Plazo)	Job Scheduler / Loader (Gutxitan, segundo/minuturo)	Multiprogramazio-maila kontrolatzen du. Diskoan dauden prozesu berrietatik zeintzuk onartuko diren RAMera erabakitzen du (Nuevo -> Listo).
Epe Ertainekoa (Medio Plazo)	Swapper (Tarteka, memoriaren arabera)	RAM memoria nagusiaren eta bigarren mailako diskoaren (Swap) arteko oreka kudeatzen du. Esekipena (Suspensión) eta Berrekitea (Reanudación) gauzatzen ditu.
Epe Laburrekoa (Corto Plazo)	CPU Scheduler / Dispatcher (Etengabe, 10-100 ms-ro)	RAMean prest daudenen artetik (Listo) CPUa uneko unean zeini esleitu erabakitzen du (Round-Robin, Lehentasunak, FCFS...).

1.3. Prozesuen 7 Egoera Posibleak eta Bizitza-Zikloa

Enuntziatuan ageri den 'Figura 1.11 Planificación de procesos' diagramak 7 egoerako eredu hedatua aurkezten du. Eredu honek 5 egoera klasikoak (Berria, Prest, Exekuzioan, Blokeatua eta Amaitua) eta memoria-esekipenaren 2 egoera osagarriak (Esekia Prest eta Esekia Blokeatua) barneratzen ditu.



2. Irudia: Enuntziatuko 'Figura 1.11 Planificación de procesos' diagramaren xehetasuna.

Egoera	Kokapen Fisikoa	Definizio Zehatza
1. Berria (Nuevo)	Diskoa / RAM hasiera	Prozesua sortu berria da; SEak bere PCB prestatzen du baina oraindik ez da onartu memorian exekutatzeko.
2. Prest (Listo)	Memoria Nagusia (RAM)	Prozesua RAMean dago, baliabide guztiak eskuragarri ditu eta CPU txandaren zain dago ilaran.
3. Exekuzioan (En ejecución)	CPU + RAM	Prozesuak CPUaren kontrol osoa dauka une horretan eta bere instrukzioak exekutatzen ari da.
4. Blokeatua (Bloqueado)	Memoria Nagusia (RAM)	Kanpoko gertaera baten zain dago (diskoan irakurri, sareko datuak jaso, tekla sakatu). Ezin du CPUa erabili.
5. Esekia Prest (Suspendido listo)	Memoria Sekundarioa (Swap)	Prozesua exekutatzeko prest dago (ez dago blokeatuta), baina RAMean lekurik ez zegoenez aldi baterako diskora eraman da.
6. Esekia Blokeatua (Suspendido bloqueado)	Memoria Sekundarioa (Swap)	Prozesua kanpoko gertaera baten zain dago ETA, aldi berean, RAM memoria askatzeko diskora eraman da.
7. Amaitua (Terminado)	Askatua	Prozesuak bere lana amaitu du (arrakastaz edo salbuespenez). SEak bere baliabide guztiak birziklatu eta PCB ezabatzen du.

Egoeren Arteko Trantsizio Gakoak:

- **Admitido (Onartua):** Nuevo -> Listo. Epe luzeko planifikatzaileak prozesua RAM memorian kargatzen du.
- **Planificación / Despacho:** Listo -> En ejecución. Epe laburreko planifikatzaileak CPUa esleitzen dio prozesuari.
- **Interrupción (Quantum agortzea):** En ejecución -> Listo. Multiataza aproposan prozesuari esleitutako denbora-tartea amaitu da; CPUa kentzen zaio.
- **E/S o espera de evento:** En ejecución -> Bloqueado. Prozesuak baliabide motela eskatu du eta itxarotera pasatzen da CPUa libre utziz.
- **Finalización E/S:** Bloqueado -> Listo. Datuak iritsi dira; prozesua prest dago CPUa berriro hartzeko.
- **Suspensión / Reanudación (Swap):** RAM memoria betetzean prozesuak diskora (Swap) bidaltzen dira (Suspensión), eta lekua dagoenean berreskuratzen dira (Reanudación).
- **Finalización E/S esekita:** Suspendido bloqueado -> Suspendido listo. Gertaera gertatu da prozesua diskoan zegoen bitartean; prest geratzen da RAMera igotzeko zain.

2. Prozesuen Arteko Komunikazioa (IPC)

Enuntziatua: *Nola ematen da prozesuen arteko komunikazioa?*

2.1. Zergatik Behar Da IPC? (Memoria-Babesa eta Isolamendua)

Sistema Eragile modernoek memoria-babes zorrotza (Memory Protection) ezartzen dute. Prozesu bakoitzari helbide-espazio birtual independente eta isolatu bat esleitzen zaio. Prozesu batek beste baten memoria atzitzen saiatzen bada, hardwareak (MMU) eta kernelak salbuespen larri bat jaurtitzen dute (Segmentation Fault Linuxen edo Access Violation Windowsen) eta prozesu erasotzailea berehala hiltzen dute.

Hala ere, aplikazio moderno askotan ezinbestekoa da prozesuek elkarrekin kolaboratzea: datu-base zentralak atzitzeko, kalkuluak paraleloan bizkortzeko edo zerbitzu modularrak eraikitzeko (adib. Web zerbitzaria + Datu-basea). Horretarako, Sistema Eragileak IPC (Inter-Process Communication) mekanismo ofizialak eskaintzen ditu.

2.2. IPC Mekanismo Nagusien Azterketa

1. Memoria Partekatua (Shared Memory): Bi prozesuk RAM memoria-segmentu fisiko berbera mapatzen dute beren helbide-espazio birtualean. Mekanismorik azkarrena da, datuak irakurri eta idazteko ez baitago sistema eragilearen bitartekaririk (Zero-Copy). Dena den, datuen koherentzia bermatzea aplikazioaren ardura da (Mutex-ak edo Semaforoak beharrezkoak dira).

2. Tutuak (Pipes): Noranzko bakarreko (unidirekzionalak) byte-korronteak dira. Tutu Anonimoak guraso-ume erlazioa duten prozesuen artean erabiltzen dira (adib. kontsolan: ps aux | grep java). Tutu Izendunak (FIFOak), ordea, fitxategi-sisteman izen bat dute eta erlaziorik gabeko prozesuen artean erabil daitezke.

3. Mezu-Ilarak (Message Queues): Sistema eragilearen nukleoan kudeatzen diren ilara sinkronizatuak dira. Izaera asinkronoa dute: igorleak mezu egituratua ilaran uzten du eta jarraitzen du; hartzaileak bere erritmoan jasotzen du.

4. Seinaleak (Signals): Gertaera asinkronoen jakinarazpen laburrak dira (software-etenaldiak). Ez dute datu-bloke handirik garraiatzen, aginduak baizik (adibidez: SIGINT prozesua eteteko, SIGTERM itxiera adeitsurako, SIGKILL berehala akabatzeko).

5. Socket-ak (Sockets): Muturreko komunikazio-kanalak dira (IP helbidea + Portua). Ordenagailu bereko prozesuen artean (Unix Domain Sockets) zein sare osoan zehar (TCP/IP edo UDP) komunikatzeko balio dute. Bezero-zerbitzari arkitekturaren oinarria dira.

6. Urruneko Prozedura-Deiak (RPC / Java RMI): Prozesu batek beste makina batean dagoen funtzio edo objektu bati dei egiten dio, deia tokikoa balitz bezala tratatuz.

2.3. Sinkronizazioa eta Problemak (Mutex, Deadlock)

Memoria partekatua edo baliabide arruntak erabiltzean, prozesuak sinkronizatu egin behar dira. Bi prozesuk aldi berean aldagai bera aldatzen badute, Lasterketa-Baldintza (Race Condition) gertatzen da. Hori saihesteko Atal Kritikoa babestu behar da Mutex (Mutual Exclusion) edo Semaforoen bidez.

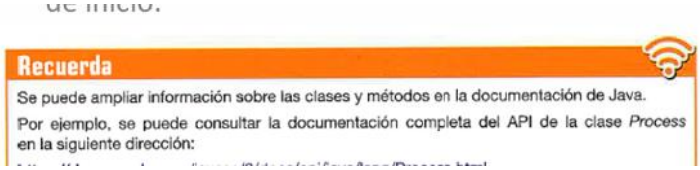
Kontu handiz ibili behar da Blokeo Elkarrekikoarekin (Deadlock): 1. prozesuak A baliabidea dauka eta B behar du, eta 2. prozesuak B dauka eta A behar du. Biak betiko geratzen dira zain, sistema osoa blokeatuz.

Mekanismoa	Abiadura	Sinkronizazioa	Eremua (Scope)	Datu Mota
Memoria Partekatua	Oso Handia (Max)	Kanpokoa (Mutex)	Makina Lokala	Datu-egitura gordinak
Tutu Anonimoak	Handia	Integratua (FIFO)	Guraso-Umeak	Byte-korrontea
Tutu Izendunak (FIFO)	Handia	Integratua (FIFO)	Makina Lokala	Byte-korrontea
Mezu-Ilarak	Ertaina	Integratua (Asink)	Makina Lokala	Mezu formatudunak
Seinaleak	Oso Azkarra	Ez da behar	Makina Lokala	Gertaera-zenbakia
Socket-ak	Ertaina (Net)	Integratua (TCP)	Lokala eta Sarea	Byteak / Paketeak / JSON

3. Java Runtime eta ProcessBuilder Klaseak

Enuntziatua: *Aztertu JAVAko Runtime eta ProcessBuilder claseak. Zertarako balio dute? Ze metodok dituzte?*

Java aplikazioak Java Makina Birtualean (JVM) isolatuta exekutatzen badira ere, maiz Sistema Eragilearen baliabide natiboekin komunikatu edo kanpoko programak abiarazi behar dituzte. Horretarako Java APIak java.lang paketean bi klase nagusi eskaintzen ditu: Runtime eta ProcessBuilder.



3. Irudia: Enuntziatuko oharra: Oracle Java Process APIaren dokumentazio ofiziala.

3.1. java.lang.Runtime Klasea

Zertarako balio du? Java aplikazio bati uneko exekuzio-inguruneareKin (JVM eta azpiko sistema eragilea) elkarrekintzan aritzeko bidea ematen dio. Singleton diseinu-patroia erabiltzen du: eraikitzailea pribatua da eta Runtime.getRuntime() metodo estatikoaren bidez eskuratzen da instantzia bakarra.

Runtime Metodoa	Funtzioa eta Deskribapen Teknikoa
getRuntime() (static)	Uneko JVMari lotutako Runtime instantzia bakarra itzultzen du.
exec(String command)	Sistema eragilearen komando bat exekutatzen du eta Process instantzia bat itzultzen du.
exec(String[] cmdarray)	Komandoa eta argumentuak array bidez pasata prozesua abiarazten du.

availableProcessors()	JVMrentzat eskuragarri dauden CPU prozesadore logikoen/nukleoen kopurua itzultzen du.
freeMemory() / totalMemory()	JVM memorian libre dagoen byte kopurua eta uneko memoria osoa itzultzen ditu.
maxMemory()	JVMak erabil dezakeen gehienezko memoria muga (-Xmx) byteetan itzultzen du.
gc()	JVMari zabor-biltzailea (Garbage Collector) exekutatzeko iradokizuna egiten dio.
exit(int status)	Uneko JVMren exekuzio osoa amaitu eta itzali egiten du (System.exit()-ek deitzen duena).

3.2. java.lang.ProcessBuilder Klasea

Zertarako balio du? Java 5 bertsioan gehitu zen Runtime.exec() zaharra ordezkatzeko. Kanpoko sistema eragileko prozesuak konfiguratu, pertsonalizatu eta abiarazteko bide moderno eta gomendatua da. Builder diseinu-patroia erabiltzen du: lehenik parametro guztiak konfiguratzen dira (ingurune-aldagaiak, direktorioa, sarrera/irteerak) eta amaieran start() deituz prozesua abiarazten da.

ProcessBuilder Metodoa	Funtzioa eta Deskribapena
command(String... / List)	Exekutatu beharreko programa eta haren argumentuak ezartzen ditu.
directory(File dir)	Prozesu berriaren lan-direktorioa (Working Directory) finkatzen du.
environment()	Prozesuaren ingurune-aldagaien mapa (Map<String, String>) itzultzen du aldaketak egiteko.
redirectInput(File / Redirect)	Sarrera estandarra (stdin) fitxategi batetik irakurtzeko birbideratzen du.
redirectOutput(File / Redirect)	Irteera estandarra (stdout) fitxategi batera idazteko birbideratzen du.
redirectErrorStream(boolean)	Egia jarriz gero, irteera estandarra eta erroreak kanal bakarrean bateratzen ditu.
inheritIO()	Prozesu berriak gurasoaren (Java programaren) kontsola S/I bera heredatzen du.
start()	Konfiguratutako prozesua sistema eragilean abiarazten du eta Process instantzia itzultzen du.

3.3. java.lang.Process eta ProcessHandle Klaseak

ProcessBuilder.start() deitzean Process objektu bat itzultzen da, martxan jarritako kanpoko prozesua kontrolatzeko:

• waitFor(): Prozesua amaitu arte itxaroten du eta irteera-kodea (exit code) itzultzen du.

• exitValue(): Amaitutako prozesuaren kodea itzultzen du (0 arrakasta da).

• isAlive(): Prozesua une honetan martxan jarraitzen duen ala jada amaitu den adierazten du.

• destroy() eta destroyForcibly(): Prozesua adeitsuki itzali (SIGTERM) edo bortxaz hiltzeko (SIGKILL).

• pid() (Java 9+): Prozesuaren PID zenbaki erreala eskuratzen du zuzenean (lehen C/JNA deiak behar ziren).

• toHandle().info().startInstant() (Java 9+): Sistema eragilearen kernelak erregistratutako hasiera-une zehatza lortzen du.

4. Exekutagarrien Zerrenda Abiarazteko Java Programa

Enuntziatua: *Se pide un programa que reciba por la línea de comandos una lista de programas ejecutables y los ejecute, mostrando su PID y el momento de inicio.*

4.1. Programaren Arkitektura eta Diseinua

Ariketak eskatzen duen programa osoa garatu da ataza2 paketean: ProzesuExekutatzailea.java. Programak honako ezaugarri sendoak barneratzen ditu:

1. Komando-lerroko argumentuak jasotzea (String[] args): Erabiltzaileak pasatako exekutagarrien zerrenda irakurtzen du.

2. Kodean bertan gordetako zerrenda lehenetsia: Argumenturik pasatzen ez bada (args.length == 0), programak ez du hutsik egiten; kodean bertan gordetako 4 aplikazio (Notepad, Kalkulagailua, Paint eta CMD) abiarazten ditu automatikoki.

3. Espazioen araberako banaketa (split("\\s+")): Komando bakunak (notepad.exe) zein parametrodunak ("cmd.exe /c start...") ondo bereizten ditu ProcessBuilder-entzat.

4. PID eta Hasiera-unearen doitasuna: Java 9ko process.pid() eta process.info().startInstant() erabiliz.

5. Errore-kudeaketa: Zerrendan fitxategi ezezagun bat badago, IOException harrapatzen du eta gainerakoekin aurrera jarraitzen du.

4.2. Kode Iturburu Osoa (ProzesuExekutatzailea.java)

```
package ataza2;

import java.io.IOException;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;

public class ProzesuExekutatzailea {
```

```
    private static final DateTimeFormatter FORMATZAILEA =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.SSS");

    // Kodean bertan gordetako programa-zerrenda lehenetsia (args hutsik badago erabiltzeko)
    private static final String[] LEHENETSITAKO_PROGRAMAK = {
            "notepad.exe",                                      // Bloc de notas
            "calc.exe",                                         // Kalkulagailua
            "mspaint.exe",                                      // Paint
            "cmd.exe /c start cmd.exe /k echo Kaixo_Mundua"     // CMD leiho ikusgaia
    };

    public static void main(String[] args) {
        System.out.println("================================================================================");
        System.out.println("       PROZESU EXEKUTATZAILEA - ZERBITZU ETA PROZESUEN PROGRAMAZIOA             ");
        System.out.println("================================================================================");

        String[] exekutagarriak;
        if (args.length > 0) {
            exekutagarriak = args;
            System.out.println("Komando-lerroko 'args' bidez jasotako programak: " + exekutagarriak.length);
        } else {
            System.out.println("[OHARRA] Ez da programarik zehaztu agindu-lerroan.");
            System.out.println("-> 'ProzesuExekutatzailea' klaseko zerrenda lehenetsia erabiliko da!");
            exekutagarriak = LEHENETSITAKO_PROGRAMAK;
        }

        List<Process> abiarazitakoProzesuak = new ArrayList<>();

        for (int i = 0; i < exekutagarriak.length; i++) {
            String komandoa = exekutagarriak[i].trim();
            System.out.println("[" + (i + 1) + "/" + exekutagarriak.length + "] Abiarazten: " + komandoa);

            try {
                String[] zatiak = komandoa.split("\\s+");
                ProcessBuilder pb = new ProcessBuilder(zatiak);
                LocalDateTime tokikoHasieraUnea = LocalDateTime.now();

                Process prozesua = pb.start();
                abiarazitakoProzesuak.add(prozesua);

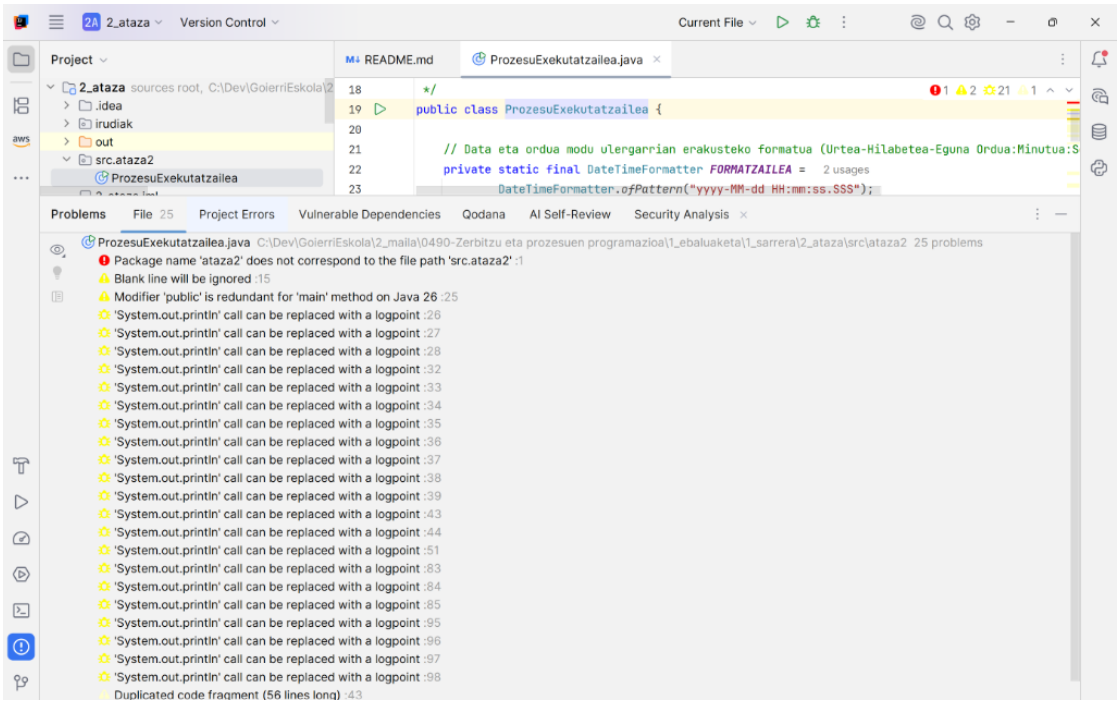
                long pid = prozesua.pid();
                ProcessHandle.Info info = prozesua.info();
                String hasieraTestua;
                if (info.startInstant().isPresent()) {
                    Instant instant = info.startInstant().get();
                    LocalDateTime seHasiera = LocalDateTime.ofInstant(instant, ZoneId.systemDefault());
                    hasieraTestua = seHasiera.format(FORMATZAILEA) + " (OS Kernel timestamp)";
                } else {
                    hasieraTestua = tokikoHasieraUnea.format(FORMATZAILEA) + " (Java Instant)";
                }

                System.out.println("  [ARRAKASTA] PID: " + pid + " | Hasiera: " + hasieraTestua);
            } catch (IOException e) {
                System.err.println("  [ERROREA] Ezin izan da exekutatu: " + e.getMessage());
            }
        }
    }
}
```

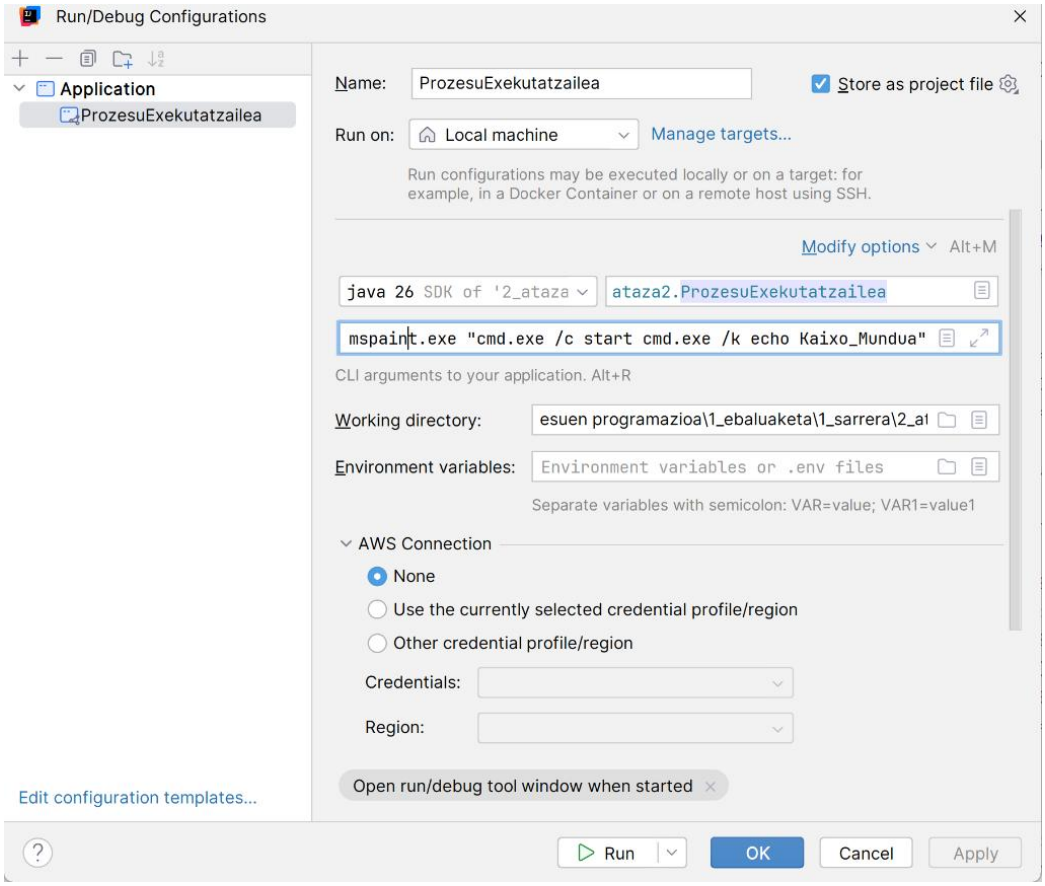
Kodea: src/ataza2/ProzesuExekutatzailea.java fitxategiaren egitura nagusia.

4.3. IntelliJ IDEA-n Konfigurazioa eta Kodearen Pantaila-Kapturak

Ariketa garatzeko eta probatzeko IntelliJ IDEA 2026 ingurunea erabili da. Jarraian, garapen-prozesuaren eta konfigurazioaren benetako pantaila-kapturak aurkezten dira:



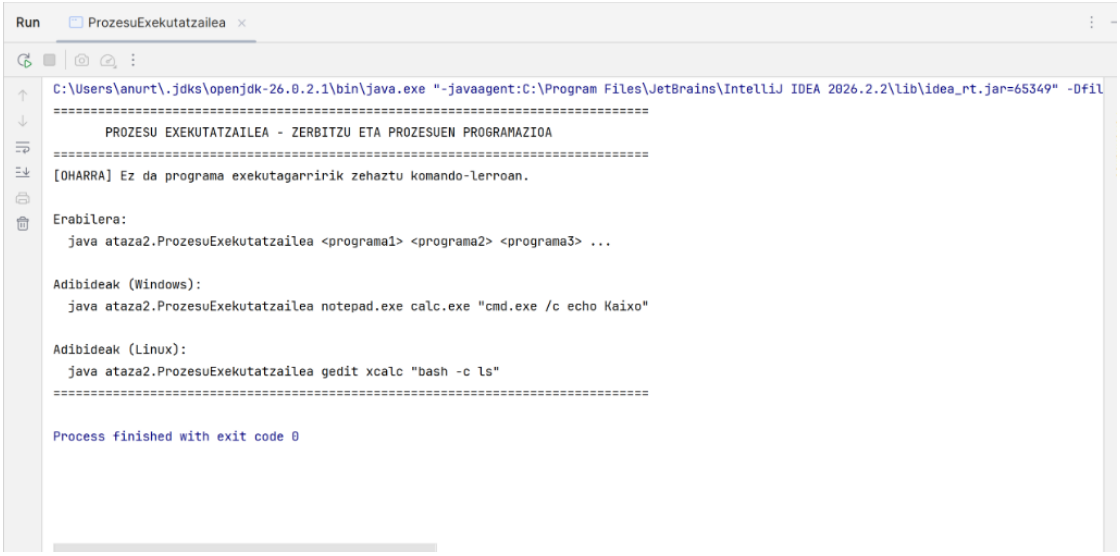
4. Irudia: ProzesuExekutatzailea.java IntelliJ IDEA editorean irekita (src/ataza2 paketean).



5. Irudia: IntelliJ-ko Run/Debug Configuration leihoa, programa-argumentuak CLI bidez konfiguratuta.

4.4. Exekuzio Errealaren Emaitzak eta Kontsolako Kapturak

Programak IntelliJ IDEA kontsolan emandako emaitza erreala, 4 programak aldi berean arrakastaz abiaraziz:



6. Irudia: IntelliJ IDEA-ko 'Run' kontsolaren pantaila-kaptura, programak nola exekutatzen diren erakutsiz.

```
================================================================================
```

```
        PROZESU EXEKUTATZAILEA - ZERBITZU ETA PROZESUEN PROGRAMAZIOA
================================================================================
Komando-lerroko 'args' bidez jasotako programak erabiliko dira.
Jasotako programa kopurua: 4
Exekuzioari hasiera ematen...

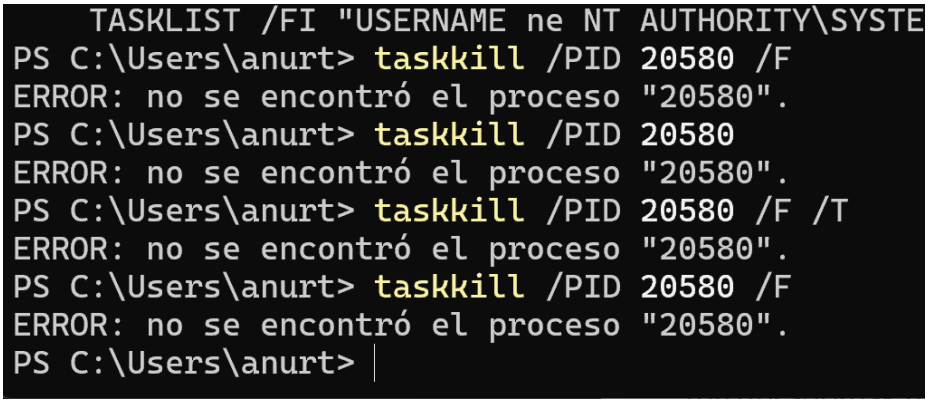
--------------------------------------------------------------------------------
[1/4] Abiarazten: "notepad.exe"
  [ARRAKASTA] Prozesua ondo abiarazi da.
  -> PID (Process ID):   16916
  -> Hasiera-unea:       2026-09-30 13:58:47.685 (OS Kernel timestamp)
  -> Bizirik dago?       BAI (Martxan)
--------------------------------------------------------------------------------
[2/4] Abiarazten: "calc.exe"
  [ARRAKASTA] Prozesua ondo abiarazi da.
  -> PID (Process ID):   11788
  -> Hasiera-unea:       2026-09-30 13:58:47.708 (OS Kernel timestamp)
  -> Bizirik dago?       BAI (Martxan)
--------------------------------------------------------------------------------
[3/4] Abiarazten: "mspaint.exe"
  [ARRAKASTA] Prozesua ondo abiarazi da.
  -> PID (Process ID):   27060
  -> Hasiera-unea:       2026-09-30 13:58:47.745 (OS Kernel timestamp)
  -> Bizirik dago?       BAI (Martxan)
--------------------------------------------------------------------------------
[4/4] Abiarazten: "cmd.exe /c start cmd.exe /k echo Kaixo_Mundua"
  [ARRAKASTA] Prozesua ondo abiarazi da.
  -> PID (Process ID):   22604
  -> Hasiera-unea:       2026-09-30 13:58:47.898 (OS Kernel timestamp)
  -> Bizirik dago?       BAI (Martxan)
--------------------------------------------------------------------------------

Prozesu guztiak abiarazi dira.
Guztira arrakastaz martxan jarritako prozesuak: 4/4
================================================================================
```

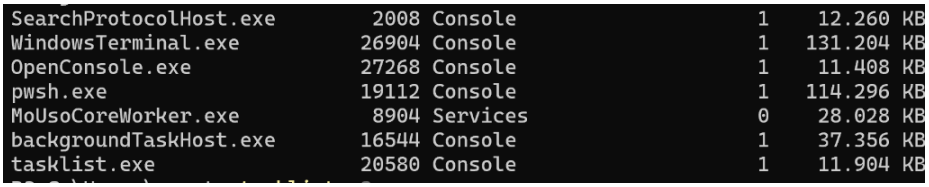
Irteera: ProzesuExekutatzailea-k emandako kontsolako testu erreala.

4.5. Kasu Praktiko Osagarria: Prozesuak Terminalean Identifikatu eta Hiltzea (tasklist & taskkill)

Praktika osagarri gisa, abiarazitako prozesuak Windows terminalean bilatu eta amaitzeko probak egin dira. Hasieran taskkill /PID 20580 /F exekutatzean 'no se encontró el proceso' errorea gertatu zen, 20580 prozesua tasklist.exe bera zelako (milisegundotan amaitzen den prozesu iragankorra).



7. Irudia: taskkill saiakeraren errorea (prozesu iragankorra amaituta zegoelako).



8. Irudia: tasklist exekuzioaren emaitza, non PID 20580 tasklist.exe bera zela ikusten den.

> 📌 **PROZESUAK HILTZEKO ONDORIOA:** Prozesu bat taskkill bidez behar bezala hiltzeko, prozesu iraunkor bat (notepad.exe adibidez) erabili behar da: 1) notepad.exe ireki; 2) tasklist bidez bere PID erreala bilatu (adib. 16916); 3) taskkill /PID 16916 /F agindua exekutatu eta berehala itxiko da.

5. ONDORIO NAGUSIAK ETA LABURPENA

1. Prozesuen kudeaketak sistema eragilearen muina osatzen du. PCB egiturari, 3 planifikazio mailei eta 7 egoerei esker, SEak baliabideen oreka zorrotza mantentzen du.

2. Prozesuak erabat isolatuta daude segurtasunagatik. Horregatik, haien arteko komunikazioak IPC mekanismo egokiak eskatzen ditu (Memoria partekatua azkartasunerako, Socketak sareko zerbitzuetarako).

3. Java plataforman, ProcessBuilder klaseak malgutasun osoa eskaintzen du kanpoko prozesuak konfiguratzeko, eta Java 9tik aurrera Process.pid() eta ProcessHandle bidez informazio natiboa modu estandarrean eskuratzen da.

6. Erreferentzia Bibliografikoak

• Oracle Java Documentation: https://docs.oracle.com/javase/8/docs/api/java/lang/Process.html

• Oracle Java 17 ProcessBuilder API: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ProcessBuilder.html

• Silberschatz, A., Galvin, P. B., & Gagne, G. (2018): Operating System Concepts (10th ed.). Wiley.

• Goierri Eskola: 0490 - Zerbitzu eta Prozesuen Programazioa material didaktikoa.